# Operating Systems Project

# Phase 1 Report

Group Members:

Yixuan Ma ym3116

Uditi Sharma us2133

# 1 Architecture and Design

First uses get_line() to get user input (main.c)

Then calls parse_line(): (parse.c)

1. Split commands into tokens
2. Syntax validation and report error
3. Build Pipeline structure with Command array (cmds)

After parsing call execute_pipeline(): (exec.c)

1. Create pipes: n_cmds - 1 pipes using create_pipes() (pipe.c)
2. Fork child processes: One child per command
3. In each child process:
    a. connect_pipes_for_child() (pipe.c) - Wire pipe ends to stdin/stdout
    b. apply_redirections() (redir.c) - Open files and redirect <, >, 2>
    c. execvp() - Replace child process with actual program
4. In parent process:
    a. Close all pipe file descriptors
    b. waitpid() - Wait for all children to finish

Finally free allocated memory and repeat

Below is an example of the whole logic:

```
  main.c:main()
      │
      ├─ Read user input ("ls -l | grep myshell > output.txt")
      │
      └─ parser.c:parse_line()
            │
            ├─ tokenize() → ["ls", "-l", "|", "grep", "myshell", ">",
"output.txt"]
            ├─ Validate syntax
            ├─ build_argv() for each command
            └─ Return Pipeline structure
      │
      └─ exec.c:execute_pipeline()
            │
            ├─ pipe.c:create_pipes() → Create 1 pipe
            │
            ├─ Fork child 1:
            │    ├─ pipe.c:connect_pipes_for_child() → Connect stdout to pipe
            │    ├─ redir.c:apply_redirections() → No redirections
            │    └─ execvp("ls", ["ls", "-l"]) → Replace with ls program
            │
            ├─ Fork child 2:
            │    ├─ pipe.c:connect_pipes_for_child() → Connect stdin to pipe
            │    ├─ redir.c:apply_redirections() → Redirect stdout to output.txt
```

```
            │     └─ execvp("grep", ["grep", "myshell"]) → Replace with grep
program
            │
            ├─ Parent closes pipes
            └─ Wait for both children
      │
      └─ free_pipeline() → Clean up memory
      │
      └─ Loop back to prompt
```

# 2 Implementation Highlights

## 2.1 Parse_line()

**1) Tokenization**: The tokenizer handles special operators (<, >, 2>, |) as distinct tokens even without spaces.

Key snippet (parser.c:114-125):

```c
  // Recognize operator: 2>
  if (*p == '2' && *(p + 1) == '>') {
      if (push_token(&tokens, &ntok, &cap, p, 2) != 0) goto oom;
      p += 2;
      continue;
  }

  // Recognize single-char operators: < > |
  if (*p == '<' || *p == '>' || *p == '|') {
      if (push_token(&tokens, &ntok, &cap, p, 1) != 0) goto oom;
      p += 1;
      continue;
  }
```

**2) Pipeline segmentation:** Split tokens by the pipe operator

The parser counts the number of segments (n_cmds = number_of_pipes + 1), allocates a Pipeline object with Pipeline.cmds as an array of Command structs, and then processes each segment independently.

**3) Per-segment parsing:** extract redirections and build argv

Build_argv() constructs the NULL-terminated argument array by skipping operators and filenames.

Key snippet (parser.c:184-200):

```
int k = 0;
 for (int i = start; i < end; i++) {
     // Skip redirection operators and their filenames
     if (strcmp(tokens[i], "<") == 0 ||
         strcmp(tokens[i], ">") == 0 ||
         strcmp(tokens[i], "2>") == 0) {
         i++; // skip filename
         continue;
     }
     if (strcmp(tokens[i], "|") == 0) continue;

     argv[k] = strdup(tokens[i]);  // Copy argument
     k++;
 }
 argv[k] = NULL;  // NULL-terminate for execvp()
```

Example: grep pattern > output.txt → argv = ["grep", "pattern", NULL]
The > output.txt is stored in Command.out_file, not in argv.

**4) Syntax validation and error reporting:** During parsing, the shell validates common syntax errors before execution. This prevents executing invalid pipelines like | ls, ls |, or ls | | grep.
Key Snippets:

```
 // Cannot start with '|'
 if (strcmp(tokens[0], "|") == 0) {
     if (err && err_sz > 0) snprintf(err, err_sz, "Command missing after pipe.");
     goto fail;
 }

 // Cannot end with '|'
 if (strcmp(tokens[ntok - 1], "|") == 0) {
     if (err && err_sz > 0) snprintf(err, err_sz, "Command missing after pipe.");
     goto fail;
 }

 // Cannot have '| |' (consecutive pipes)
 for (int i = 0; i < ntok - 1; i++) {
     if (strcmp(tokens[i], "|") == 0 && strcmp(tokens[i + 1], "|") == 0) {
         if (err && err_sz > 0) snprintf(err, err_sz, "Empty command between
pipes.");
         goto fail;
     }
 }
```

After parsing succeeds, the main loop passes the resulting structure to the execution layer.
The executor does only reads:
**pipeline.n_cmds** to determine how many processes are needed
**pipeline.cmds[**i**].argv** to run each command via execvp()
**pipeline.cmds[i].in_file/out_file/err_file** to apply redirections using open() + dup2()

## 2.2 execute_pipeline()

### 1. Four-step orchestration

`execute_pipeline()` is the single entry point called by `main()` after a successful parse. It runs four steps in order:

1. Create pipes
2. Fork one child per command
3. Parent closes pipes
4. Parent waits for all children

### 2. Pipe creation and heap allocation

The pipe array and PID array are both allocated on the heap with `malloc()` so the shell supports pipelines of any length. For `n` commands, exactly `n - 1` pipes are needed.

**Key snippet (exec.c:99–113):**

```
if (n_pipes > 0) {
  pipe_fds = malloc((size_t)n_pipes * sizeof(int[2]));
  if (pipe_fds == NULL) {
     perror("malloc (pipe_fds)");
     return -1;
  }
  if (create_pipes(n_pipes, pipe_fds) < 0) {
     free(pipe_fds);
     return -1;
  }
}
```

**3. Fork loop — child setup**

For each command, `fork()` spawns a child. Inside the child, three things happen in strict order:

a. `connect_pipes_for_child()` wires pipe ends onto `STDIN` and `STDOUT` via `dup2()`
b. `apply_redirections()` opens any `<`, `>`, or `2>` files and overrides the standard descriptors
c. `execvp()` replaces the child image with the actual program

**Key snippet (exec.c:156–193):**

```
if (n_pipes > 0) {
  connect_pipes_for_child(i, n_cmds, n_pipes, pipe_fds);
}
if (apply_redirections(&p->cmds[i]) < 0) {
  exit(1);
}
execvp(p->cmds[i].argv[0], p->cmds[i].argv);

if (n_cmds == 1) {
  fprintf(stderr, "Command not found.\n");
} else {
  fprintf(stderr, "Command not found in pipe sequence.\n");
}
exit(127);
```

If `execvp()` returns at all, the command was not found. The error message differs depending on whether this is a standalone command or part of a pipeline, matching the spec exactly.

---

**4. Parent closes all pipe ends**

After all children are forked, the parent calls `close_all_pipes()` on its own copies of every pipe file descriptor. If the parent kept a write end open, the child reading from that pipe's read end would never receive EOF and would block forever.

**Key snippet (exec.c:211–214):**

```
if (pipe_fds) {
  close_all_pipes(n_pipes, pipe_fds);
  free(pipe_fds);
```

```
}
```

**5. Waiting for all children**

The parent calls `waitpid()` for every child PID, not just the last one. This prevents zombie processes and ensures output from every stage is flushed before the prompt reappears. The exit status of the last command is captured to mirror real shell `$?` behaviour.

**Key snippet (exec.c:230–243):**

```
for (int i = 0; i < n_cmds; i++) {
  int status;
  waitpid(pids[i], &status, 0);
  if (i == n_cmds - 1) {
     if (WIFEXITED(status))
        last_exit = WEXITSTATUS(status);
     else
        last_exit = 1;
  }
}
```

## 2.3 Pipe helpers (pipe.c)

**1. create_pipes()**

Opens `n_pipes` anonymous pipes sequentially. If any `pipe()` call fails, all previously opened pipes are closed before returning so no file descriptors leak to the caller.

**Key snippet (pipe.c:56–68):**

```
for (int i = 0; i < n_pipes; i++) {
  if (pipe(pipe_fds[i]) < 0) {
     perror("pipe");
     for (int j = 0; j < i; j++) {
        close(pipe_fds[j][0]);
        close(pipe_fds[j][1]);
     }
     return -1;
  }
```

}

**2. connect_pipes_for_child()**

Uses `dup2()` to wire the correct pipe ends onto `STDIN_FILENO` and `STDOUT_FILENO` based on the child's position in the pipeline. After `dup2()`, `close_all_pipes()` discards every raw pipe file descriptor in the child so they are not inherited by the exec'd program.

**Layout for `cmd0 | cmd1 | cmd2`:**

- `cmd_idx = 0` (first): stdout → `pipe_fds[0][1]`
- `cmd_idx = 1` (middle): stdin ← `pipe_fds[0][0]` and stdout → `pipe_fds[1][1]`
- `cmd_idx = 2` (last): stdin ← `pipe_fds[1][0]`

**Key snippet (pipe.c:141–163):**

```
if (cmd_idx > 0) {
  dup2(pipe_fds[cmd_idx - 1][0], STDIN_FILENO);
}
if (cmd_idx < n_cmds - 1) {
  dup2(pipe_fds[cmd_idx][1], STDOUT_FILENO);
}
close_all_pipes(n_pipes, pipe_fds);
```

**3. close_all_pipes()**

Closes both ends of every pipe in the array. Shared by the parent (after forking) and each child (after `dup2()` inside `connect_pipes_for_child()`).

## 2.4 apply_redirections() (redir.c)

Handles the three standard redirections for a single command. Each non-NULL field in the `Command` struct triggers an `open()` plus `dup2()` pair. The original file descriptor from `open()` is closed immediately after `dup2()` so it does not leak into the exec'd program.

| Token | open() flags | Installed onto |
|---|---|---|
| `<` | `O_RDONLY` | `STDIN_FILENO` |
| `>` | `O_WRONLY | O_CREAT |
| `2>` | `O_WRONLY | O_CREAT |

**Key snippet (redir.c:59–78):**

```
if (cmd->in_file != NULL) {
  int fd = open(cmd->in_file, O_RDONLY);
  if (fd < 0) {
      fprintf(stderr, "File not found.\n");
      return -1;
  }
  dup2(fd, STDIN_FILENO);
  close(fd);
}
```

**Critical design decision — ordering**

`apply_redirections()` runs after `connect_pipes_for_child()`. This gives explicit file redirections higher priority than pipe connections, which is the correct shell behaviour.

For example:

cat < input.txt | grep foo

`cat`'s stdin is first wired to the pipe by `connect_pipes_for_child()`, then immediately overridden with `input.txt` by `apply_redirections()`.

# 3 Execution Instructions

1. cd to the myshell-os-project file
2. Run “make” in terminal
3. Run “./myshell” and you have started the shell!!
4. After exiting, run “make clean” to clean up the o files

For automated testings:

1. cd to the myshell-os-project file
2. Run “make”
3. Run “./setup_tests.sh” to set up the test files
4. Run “./myshell < run_tests.txt” to start the test
5. After finishing testing, run “./cleanup_tests.sha” and “make clean” to clean up the test files and o files

# 4 Testing

1. Basic Commands & Arguments

```
(base) yixuanma@Annies-MacBook-Air myshell-os-project % ./myshell
$ pwd
/Users/yixuanma/Desktop/nyush/spring 2026(NYUAD)/OS/myshell-os-project
$ ls -l Makefile
-rw-r--r--@ 1 yixuanma  staff  293 Feb 21 17:29 Makefile
$ echo hello world
hello world
```

This tests command execution with fork() + execvp(), argument parsing

2. Pipes

```
$ cat input.txt | grep a | sort
apple
apricot
banana
```

3. Redir

```
$ cat < input.txt
apple
banana
apricot
best$
$ ls bad 2> err.txt
$ cat err.txt
ls: bad: No such file or directory
```

4. Compound

```
$ cat < input.txt | grep a > filtered.txt
$ cat filtered.txt
apple
banana
apricot
```

Finds words in input.txt that contains “a” and writes to filtered.txt

5. Syntax Errors

```
$ cat <
Input file not specified.
$ echo hello >
Output file not specified.
$ ls 2>
Error output file not specified.
$ ls |
Command missing after pipe.
$ cat | | wc
Empty command between pipes.
```

6. Runtime Errors

```
$ invalid_command
Command not found.
$ cat input.txt | invalid_cmd | wc -l
Command not found in pipe sequence.
       0
$ cat < nonexistent_file.txt
File not found.
```

# 5 Challenges

**1. Getting the pipe-close timing right**

Early in development the shell would hang indefinitely after commands like ls | cat. The cause was that the parent still held open write ends of the pipes after forking all children. cat was waiting for EOF on its stdin, but because the parent had an open write end the kernel never delivered it. Adding close_all_pipes() in the parent after the fork loop resolved this completely.

**2. Ordering of pipe wiring vs. file redirections**

Determining that connect_pipes_for_child() must run before apply_redirections() required careful reasoning. If the order were reversed, an explicit < file on a non-first command would install the file on STDIN_FILENO, and then connect_pipes_for_child() would overwrite it with the pipe's read end — the wrong result. The current order means the pipe is installed first and the explicit redirection, if present, correctly overrides it.

**3. Preventing zombie processes on partial fork failure**

If fork() fails while spawning the third child in a four-command pipeline, children 0 and 1 are already running but have no parent calling waitpid() for them — they become zombies. The error path in execute_pipeline() explicitly loops over all pids[] entries collected so far and calls waitpid() for each one before returning, preventing this.

**4. File descriptor leaks into exec'd programs**

After dup2() installs a needed pipe end onto STDIN_FILENO or STDOUT_FILENO, the original numbered file descriptor is still open. If it is not closed, the exec'd program inherits it as an unexpected open fd, which can cause pipes to never see EOF. Calling close_all_pipes() inside each child at the end of connect_pipes_for_child() ensures a clean file descriptor table before execvp().

**5. Handling the "2>" operator in the tokenizer**

A naive character-by-character tokenizer would treat 2> as the word "2" followed by the operator ">". The tokenizer in parser.c explicitly checks for the two-character sequence before checking for single-character operators, so 2> is always produced as a single token regardless of surrounding whitespace.

## 6 Division of Tasks

**Person A — Yixuan Ma (ym3116): Parser and syntax validation**

- `parser.c`: `tokenize()`, `parse_line()`, `build_argv()`, `free_pipeline()`
- `parser.h`: `Command` and `Pipeline` struct definitions; `parse_line()` and `free_pipeline()` declarations
- `errors.c`: shared error message helpers and constants

**Responsibility:** turn the raw input string into a validated `Pipeline` struct and catch all syntax errors (missing filename after a redirection operator, pipe at start/end of input, empty segment between pipes) before any execution takes place.

**Person B — Uditi Sharma (us2133): Execution engine**

- `exec.c`: `execute_pipeline()` — pipeline orchestration (`fork`, `wait`, `execvp`)
- `pipe.c`: `create_pipes()`, `close_all_pipes()`, `connect_pipes_for_child()`
- `redir.c`: `apply_redirections()`
- `exec.h`: public interface declarations for all execution functions

**Responsibility:** given a valid `Pipeline`, perform all OS-level work — creating anonymous pipes, forking one child per command, wiring file descriptors with `dup2()`, applying file redirections, calling `execvp()`, and waiting for all children. Also responsible for runtime error detection (file not found, command not found).

Both members collaborated on `main.c` (the REPL loop, built-in exit handling, and the parse → execute → free flow) and the `Makefile`.

## 7 References


1. Kerrisk, M. (2010). *The Linux Programming Interface*. No Starch Press. (Chapters 24–27: process creation, exec, pipes, and file I/O.)

2. Stevens, W. R., & Rago, S. A. (2013). *Advanced Programming in the UNIX Environment* (3rd ed.). Addison-Wesley. (Chapters 8 and 15: process control and interprocess communication.)
3. IEEE Std 1003.1-2017 (POSIX.1-2017). Open Group Base Specifications. Relevant man pages: `fork(2)`, `execvp(3)`, `pipe(2)`, `dup2(2)`, `waitpid(2)`, `open(2)`.
4. Linux man-pages project. `man fork(2)`, `man execvp(3)`, `man pipe(2)`, `man dup2(2)`, `man waitpid(2)`, `man open(2)`. https://man7.org/linux/man-pages/
5. Bryant, R. E., & O'Hallaron, D. R. (2015). *Computer Systems: A Programmer's Perspective* (3rd ed.). Pearson. (Chapter 8: Exceptional Control Flow — processes, signals, and nonlocal jumps.)